

NANYANG TECHNOLOGICAL UNIVERSITY
SINGAPORE

College of Computing and Data Science

SC2006 – Software Engineering

Lab 4 Deliverables – AI Report

SCSJ Group 3

Name	Matriculation No.
Xu Hanwen	U2522613H
Teo Kok Wei, Oscar	U2410894D
K Eswaran	U2521677A
Mao Yikai	U2520028F
Tan Leong Jun Joseph	U2321339H

(1) Tool and set-up

We used Claude Code, an in-IDE coding agent, in VS Code (model: Claude Opus 5.5), the same AI family as Labs 2 and 3. Each exercise ran in a new session with no earlier conversation, on a copy of our repository without its Git history, so the agent could only learn about the project from the files in front of it. We asked it to work only inside that copy.

(2) Exercise 3.3.2: prompt and materials

We deleted the whole of server/src/services/alternatives.ts, which holds AlternativesFinder, the control class we black-box tested in the Lab 4 deliverables. The project then no longer compiled. We pointed the agent to:

- our tests: server/tests/alternatives.test.ts and the 13 black-box cases (BB-01 to BB-13) in server/tests/alternatives.blackbox.test.ts;
- where the class is used: server/src/routes/venues.ts;
- our Lab 3 design class diagram (class-3-server.puml) and the ViewAlternatives (#3-2) use case description.

The rest of the codebase was available for it to read. We did not give it the old file with the function bodies removed.

Prompt:

The file server/src/services/alternatives.ts has been deleted. It contained AlternativesFinder, the control class for the ViewAlternatives (#3-2) use case: given the venue being viewed and the list of all venues, it recommends quieter venues nearby. Recreate this file from scratch. Use these for reference: the tests in server/tests/alternatives.test.ts and server/tests/alternatives.blackbox.test.ts, where the class is used in server/src/routes/venues.ts, our design class diagram materials/class-3-server.puml, and the use case description materials/use-case-3-2.txt. Follow the style of the other files in server/src/services. Make sure the server compiles (npx tsc --noEmit in server/) and the tests pass (npm test in server/).

Response:

The agent read the two test files, the use case, the class diagram, the route, the Venue type, the distance helper and two other services for style, then wrote the file and ran the checks itself. The server compiled, but the first test run gave 46 of 47 passing: BB-10 (a candidate exactly 3.000 km away) returned no alternative instead of one. It worked out the distances the test produces (3 km came out as 3.000000000000002 km), added a tiny tolerance to the 3 km limit, and re-ran: all 47 tests passed. The whole exercise took about a minute and a half. Its final file:

```
import { Venue } from "../types/venue";
import { haversineKm } from "./distance";

// AlternativesFinder - control class for ViewAlternatives (#3-2), which
// extends ViewVenueDetail (#3-1). Suggests quieter venues of the same
// category close to the one being viewed.
const MAX_DISTANCE_KM = 3;
const MAX_RESULTS = 3;
// Haversine maths carries a little floating-point error (a venue exactly
// 3km away can come out as 3.000000000000002km), so the 3km limit allows
// for it. 1e-9km is far below any real distance difference.
const DISTANCE_EPSILON_KM = 1e-9;

// A Closed venue reads 0% and an Unavailable one has no reading at all, so
// neither is genuinely "less crowded" - both are left out as alternatives.
function hasLiveReading(venue: Venue): boolean {
  return venue.crowdLevel !== "Closed" && venue.crowdLevel !== "Unavailable";
}

/**
 * Finds lower-crowd venues of the same category near the venue being viewed.
 * Transit venues are excluded from alternatives matching.
 * @param venue - the venue being viewed
 * @param all - every venue to choose from (may include the viewed venue itself)
 * @returns up to three alternatives within 3km with a lower crowd percentage,
 *   nearest first, each with its distanceKm from the viewed venue (empty if
 *   none qualify)
 */
export function findAlternatives(venue: Venue, all: Venue[]): Venue[] {
  if (venue.category === "Transit") return [];

  return all
    .filter(
      (other) =>
        other.id !== venue.id &&
        other.category === venue.category &&
        hasLiveReading(other) &&
        other.crowdPercent < venue.crowdPercent
    )
    .map((other) => ({
      ...other,
      distanceKm: haversineKm(venue.lat, venue.lng, other.lat, other.lng),
    }))
    .filter((other) => other.distanceKm <= MAX_DISTANCE_KM + DISTANCE_EPSILON_KM)
    .sort((a, b) => a.distanceKm - b.distanceKm)
    .slice(0, MAX_RESULTS);
}
```

(3) Reflection on 3.3.2

How we prompted it, and the materials we provided

One prompt that named the deleted class, its use case and the files to look at, and asked it to check its work by compiling and running the tests. The tests mattered most: they pin down rules the use case description leaves out.

Was any iteration needed?

No further prompting was needed. The agent iterated by itself once: when BB-10 failed, it investigated and fixed the cause rather than changing the test.

What was the outcome?

A working class that compiles and passes every test, and behaves the same as ours for every case we test. Two differences from our version: it allows a tolerance of 0.000000001 km at the 3 km limit, while we round distances to the nearest metre; and it returns no alternatives at once when the viewed venue is a station, following the use case note, while our code leaves that to the app.

The most interesting result is that it made the same mistake we did. Our own first version also excluded a venue exactly 3 km away, and BB-10 caught it in both cases. This shows the value of boundary value tests: they found the same defect in human-written and AI-written code.

Did it meet our expectations?

Yes, and it went further than we expected. We expected it to manage such a small class, given detailed tests, but not that it would diagnose a floating-point problem on its own. It also pointed out that the use case description does not mention leaving out closed venues, venues without a reading, or the limit of three results. These rules came only from the tests, so we will add them to ViewAlternatives (#3-2) in the Lab 5 SRS.

What could be done differently?

Nothing was needed for success here. A harder test of the AI would be to give it only the use case and the class diagram, without the tests; it would then have had to guess the rules above.

(4) Exercise 3.3.3: generating updated class diagrams for Lab 5

We chose the second suggested experiment: asking the AI to generate the updated class diagrams for our Lab 5 submission. Since Lab 3 we have added friends, a meetup planner with a choice of meeting time, and rate limiting, so our Lab 3 diagrams are out of date. We gave a new session our four Lab 3 diagrams as PlantUML files, and the current code.

Prompt:

Our Lab 3 design class diagrams are in materials/ (class-1-overview.puml, class-2-app.puml, class-3-server.puml, class-4-entities.puml). The code has changed since Lab 3: among other things we added friends, a meetup planner where the group can choose a meeting time, and rate limiting. Generate updated versions of these four class diagrams that match the current codebase in app/ and server/, for our Lab 5 submission. Keep the same PlantUML style, the same stereotypes (<<boundary>>, <<control>>, <<entity>> and so on) and the design pattern labels, and show only the important attributes and operations. Save them as materials/lab5/class-1-overview.puml, materials/lab5/class-2-app.puml, materials/lab5/class-3-server.puml and materials/lab5/class-4-entities.puml, and list what you changed compared with Lab 3.

Response (summarised):

The agent read about 40 source files across the app and server, wrote the four diagrams, checked that each one parses and renders, and listed its changes. It took about six minutes. The main changes:

- Entities: new Friendship, HomeStation, MeetupPlan, MeetupSuggestion, TravelTime and MeetingTime, with their associations to Account, Station and Venue.
- Server: new FriendsController, MeetupController, MeetupPlanner and RateLimiter, with links showing which routes are rate limited and how the planner uses CrowdAggregator, DirectionsController and PopularTimesService.
- App: new MeetupUI, FriendsUI and StationPickerUI boundary classes, a MeetingTimeController, and the new ApiClient operations.
- Corrections to Lab 3: Favourite.createdAt became savedAt (the database column is saved_at), and PopularTimesPattern.currentReading(now) became readingAt(time).
- Layout: it added one hidden link so the overview diagram stays within PlantUML's maximum image width.

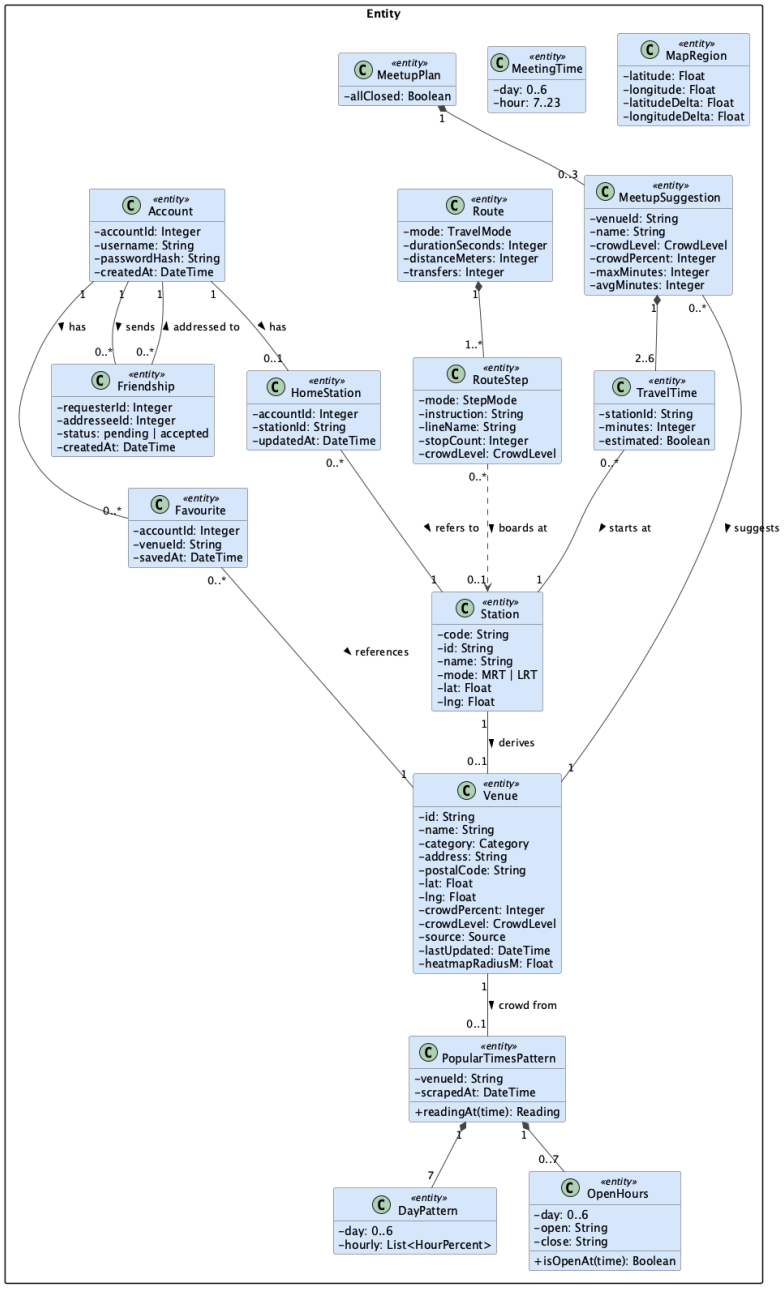


Figure 1. AI-generated Lab 5 entity class diagram

(5) Reflection on 3.3.3

How we prompted it, and the materials we provided

One prompt, with the Lab 3 diagrams as the starting point and a list of the main features added since. We asked for the same style and stereotypes so the result would fit our earlier deliverables.

Was any iteration needed?

No further prompting. The agent checked its own diagrams by rendering them and fixed the overview when it came out too wide.

What was the outcome?

Four diagrams that render, cover every new feature, and keep our Lab 3 style. We checked them against the code. The new classes and associations are correct, and both of its corrections to Lab 3 are right: our Lab 3 entity diagram had the wrong name for when a favourite was saved. We also found problems:

- Several operation names are invented. Our routes and services are plain functions, so names such as FriendsController.getOverview() do not appear in the code. The agent said so itself.
- MeetingTime.day is shown as 0..6, which reads as a day of the week; in the code it is the number of days from today.
- The server diagram is crowded, with many crossing lines, and some link labels sit away from their lines.
- RateLimiter is drawn as one class, although the code creates a separate limiter for each route.

Did it meet our expectations?

Mostly. It was faster and more thorough than updating the diagrams by hand, and it found a mistake in our Lab 3 diagram that we had missed. But it is a starting point, not a finished deliverable: we will fix the points above and tidy the server diagram's layout before using these diagrams in Lab 5.

What could be done differently?

Asking for one diagram at a time would make each easier to review. Giving it our Git history would let it tell what is genuinely new since Lab 3, rather than inferring it from the code. Telling it which names our design uses for function-based classes would avoid invented operation names.